Hệ thống phát hiện xâm nhập phân tán thời gian thực trên cụm thiết bị biên Jetson Orin Nano Super với Kafka và PySpark

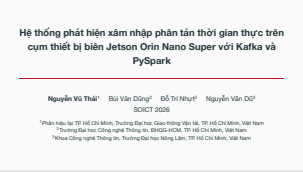
Nguyễn Vũ Thái[1] Bùi Văn Dũng[2] Đỗ Trí Nhựt[2] Nguyễn Văn Dũ[3]

SOICT 2026

[1]Phân hiệu tại TP. Hồ Chí Minh, Trường Đại học Giao thông Vận tải, TP. Hồ Chí Minh, Việt Nam
[2]Trường Đại học Công nghệ Thông tin, ĐHQG-HCM, TP. Hồ Chí Minh, Việt Nam
[3]Khoa Công nghệ Thông tin, Trường Đại học Nông Lâm, TP. Hồ Chí Minh, Việt Nam

2026-10-13

Hệ thống phát hiện xâm nhập phân tán thời gian thực trên cụm thiết bị biên Jetson Orin Nano Super với Kafka và PySpark


Hệ thống phát hiện xâm nhập phân tán thời gian thực trên cụm thiết bị biên Jetson Orin Nano Super với Kafka và PySpark

Nguyễn Vũ Thái[1] Bùi Văn Dũng[2] Đỗ Trí Nhựt[2] Nguyễn Văn Dũ[3]

SOICT 2026

[1]Phân hiệu tại TP. Hồ Chí Minh, Trường Đại học Giao thông Vận tải, TP. Hồ Chí Minh, Việt Nam
[2]Trường Đại học Công nghệ Thông tin, ĐHQG-HCM, TP. Hồ Chí Minh, Việt Nam
[3]Khoa Công nghệ Thông tin, Trường Đại học Nông Lâm, TP. Hồ Chí Minh, Việt Nam

Câu hỏi của bài: có hai bo mạch biên thì nên chia việc phát hiện xâm nhập giữa chúng thế nào, và bo thứ hai thực sự đem lại bao nhiêu.

I. VẤN ĐỀ VÀ ĐÓNG GÓP

I. VẤN ĐỀ VÀ ĐÓNG GÓP

Phát hiện xâm nhập ngay tại thiết bị biên: còn ba khoảng trống

Mô hình phát hiện xâm nhập nay đã *chạy* được trên bo mạch nhỏ, nhưng nếu một bo phải xét **mọi** luồng thì nó quá tải rất nhanh.

Nghiên cứu trước còn thiếu

1. Chỉ đo trên **một thiết bị**, không chia việc cho nhiều bo.
2. Hiếm khi báo **cùng lúc** tốc độ, độ trễ và năng lượng.
3. Điểm số offline lấy từ quy trình **rò rỉ dữ liệu**.

Câu hỏi

Có hai bo Jetson thì nên *đặt* việc phát hiện lên chúng thế nào?

Cách làm

Tách làm hai: **cổng lọc** rẻ trên bo #1, **bộ phân loại** nặng trên bo #2, nối bằng Kafka.



Khoảng trống thứ ba là lý do nhóm làm bài FAIR về quy trình đánh giá chống rò rỉ.

Các đóng góp

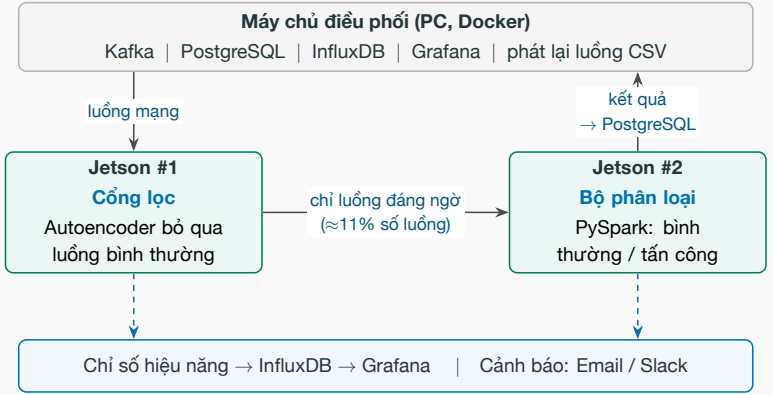
1. Hệ thống phát hiện xâm nhập **hai tầng, phân tán**: cổng lọc và bộ phân loại Spark đặt trên hai bo khác nhau.
2. **Ba cách triển khai** trên cụm hai Jetson: tách tầng, nhân bản ngang, cụm Spark.
3. **Đo đầu cuối**: tốc độ, độ trễ, CPU/RAM từng bo *và* năng lượng mỗi dự đoán.
4. Mã nguồn, cấu hình và kịch bản đo **công khai**, chạy lại được.

Các đóng góp
1. Hệ thống phát hiện xâm nhập hai tầng, phân tán: cổng lọc và bộ phân loại Spark đặt trên hai bo khác nhau.
2. Ba cách triển khai trên cụm hai Jetson: tách tầng, nhân bản ngang, cụm Spark.
3. Đo đầu cuối: tốc độ, độ trễ, CPU/RAM từng bo và năng lượng mỗi dự đoán.
4. Mã nguồn, cấu hình và kịch bản đo công khai, chạy lại được.

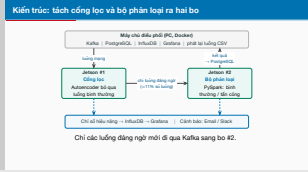
Dự đoán (verdict) ở đây là kết luận cuối cùng bình thường hay tấn công cho một luồng.

II. HỆ THỐNG VÀ CÁCH ĐO

Kiến trúc: tách cổng lọc và bộ phân loại ra hai bo



Chỉ các luồng đáng ngờ mới đi qua Kafka sang bo #2.



Máy chủ chạy hạ tầng. Bo một lọc, bo hai phân loại. Mọi chỉ số đi vào Grafana theo thời gian thực.

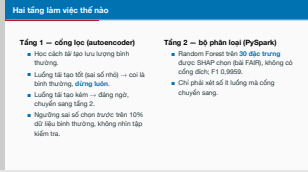
Hai tầng làm việc thế nào

Tầng 1 — cổng lọc (autoencoder)

- Học cách *tái tạo* lưu lượng bình thường.
- Luồng tái tạo tốt (sai số nhỏ) → coi là bình thường, **dừng luôn**.
- Luồng tái tạo kém → đáng ngờ, chuyển sang tầng 2.
- Ngưỡng sai số chọn *trước* trên 10% dữ liệu bình thường, không nhìn tập kiểm tra.

Tầng 2 — bộ phân loại (PySpark)

- Random Forest trên **30 đặc trưng** được SHAP chọn (bài FAIR), không có cổng đích; F1 0,9959.
- Chỉ phải xét số ít luồng mà cổng chuyển sang.



Cổng lọc dùng đủ 77 đặc trưng, ngưỡng là phân vị 0,995 của sai số trên 10% luồng bình thường của tập huấn luyện. Mỗi tầng xử lý theo lô 20 luồng, hoặc sớm hơn khi luồng cũ nhất đã chờ 1 giây.

Vì sao bộ phân loại là Random Forest trên Spark

- **Random Forest có sẵn trong Spark ML**, nên mô hình xuất ra chạy được trên Jetson (ARM64) mà không cần thư viện native như XGBoost.
- **Ít đặc trưng** (30 thay vì 77) thì dựng vector và suy luận nhanh hơn trên bo 8 GB.
- **Cùng một bộ công cụ** (Spark) vừa huấn luyện trên cụm vừa chạy trên thiết bị — cái giá của lựa chọn này ở Kết quả 5.

Vì sao bộ phân loại là Random Forest trên Spark
- Random Forest có sẵn trong Spark ML, nên mô hình xuất ra chạy được trên Jetson (ARM64) mà không cần thư viện native như XGBoost.
- Ít đặc trưng (30 thay vì 77) thì dựng vector và suy luận nhanh hơn trên bo 8 GB.
- Cùng một bộ công cụ (Spark) vừa huấn luyện trên cụm vừa chạy trên thiết bị — cái giá của lựa chọn này ở Kết quả 5.

Đây đúng là mô hình SHAP Top-30 của bài FAIR, huấn luyện lại trên toàn bộ tập train: F1 0,9959 (bài FAIR báo 0,9955).

Các cách triển khai và cách đo

Một bo làm hết mọi việc, tắt hoặc bật cổng lọc.

A: Tách tầng cổng trên #1, phân loại trên #2.

B: Nhân bản mỗi bo làm đủ cả hai việc (tắt cổng).

C: Cụm Spark PC làm master, hai bo làm worker.

Theo dõi **từng luồng từ lúc gửi tới lúc có kết quả** tính cả thời gian chờ. **Chịu được**: mức tải cao nhất mà mọi luồng có kết quả trong vòng 10 giây sau khi ngừng tải.

Thiết lập đo

2× Jetson Orin Nano Super: CPU 6 nhân, 8 GB RAM

PySpark 3.4.1, Kafka 3.5, mạng Wi-Fi

10.000 luồng từ tập kiểm tra, 15% tấn công

Quét mức tải 1–40 luồng/giây, mỗi mức 2 lần

Năng lượng đo bằng `tegrastats`



Mỗi dự đoán lưu thời điểm gửi của luồng, nên độ trễ tính cả thời gian nằm trong hàng đợi. Mỗi lần chạy dùng một consumer group Kafka riêng để không xử lý phần tồn của lần trước.

III. KẾT QUẢ VÀ KẾT LUẬN

Kết quả 1: tách tầng chịu được tải gấp 9 lần

Cách triển khai	Chịu được (luồng/giây)	p95 (giây)
Một bo	2,5	11,3
Một bo + cổng	≈2,5	7,8
B: Nhân bản	3,3	8,2
C: Cụm Spark	9–17	5,2
A: Tách tầng	**22,4**	**7,1**

p95: 95% số luồng có kết quả nhanh hơn mức này, tính từ lúc gửi, gồm cả thời gian chờ.

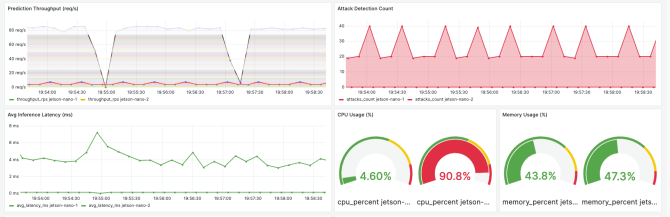
- **Tách tầng: gấp 9 lần** một bo.
- **Cùng cổng mà để chung một bo thì không nhanh hơn**: lợi ích đến từ bo thứ hai.
- Độ trễ vài giây vì mỗi lô Spark mất vài giây.

Nên dùng tách tầng: chịu được tải cao nhất.



Chế độ một bo và nhân bản tắt cổng, như bản đã nộp. Độ trễ trung vị của tách tầng khoảng 1 giây. Cổng bỏ qua khoảng 89% số luồng ở mọi chế độ có bật cổng.

Kết quả 2: nhìn trực tiếp thấy bo nào đang gánh việc

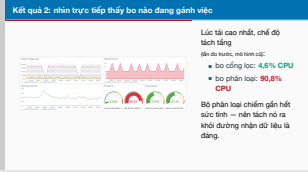


Lúc tải cao nhất, chế độ tách tầng

(lần đo trước, mô hình cũ):

- bo cổng lọc: **4,6% CPU**
- bo phân loại: **90,8% CPU**

Bộ phân loại chiếm gần hết sức tính — nên tách nó ra khỏi đường nhận dữ liệu là đúng.



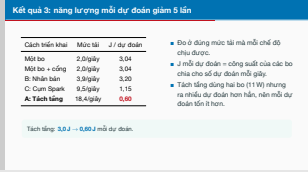
Ảnh Grafana chụp khi đang phát lại dữ liệu ở chế độ tách tầng.

Kết quả 3: năng lượng mỗi dự đoán giảm 5 lần

Cách triển khai	Mức tải	J / dự đoán
Một bo	2,0/giây	3,04
Một bo + cổng	2,0/giây	3,04
B: Nhân bản	3,9/giây	3,20
C: Cụm Spark	9,5/giây	1,15
A: Tách tầng	18,4/giây	**0,60**

- Đo ở đúng mức tải mà mỗi chế độ chịu được.
- J mỗi dự đoán = công suất của các bo chia cho số dự đoán mỗi giây.
- Tách tầng dùng hai bo (11 W) nhưng ra nhiều dự đoán hơn hẳn, nên mỗi dự đoán tốn ít hơn.

Tách tầng: **3,0 J** → **0,60 J** mỗi dự đoán.



Năng lượng đo bằng tegrastats: 30 giây nền rồi cửa sổ tải; chế độ hai bo cộng cả hai bo. Khi chạy, công suất tăng thêm khoảng 1,5 W ở mọi chế độ.

Kết quả 4: cổng lọc cho lọt bao nhiêu tấn công

Ngưỡng cổng (phân vị)	Tấn công chuyển tiếp	Báo nhầm	Luồng bỏ qua
0,99	72,4%	0,99%	88,3%
0,995 (đang dùng)	**68,1%**	**0,49%**	89,4%
0,999	39,8%	0,09%	94,0%

Offline trên tập kiểm tra: cổng dùng đủ 77 đặc trưng. Chạy thật: bắt được 69,6% trong 1.261 luồng tấn công.

- Luồng bị cổng bỏ qua là *chốt luôn*: **khoảng 30% luồng tấn công** không tới được bộ phân loại.
- Nếu cổng chỉ dùng 30 đặc trưng của bộ phân loại thì chỉ chuyển tiếp được 44,8%.
- Người vận hành chọn điểm hợp với sức máy.

Tải gấp 9 lần phải trả bằng recall: cổng là một **sự đánh đổi có thể chỉnh**, không phải bộ lọc miễn phí.



Khi tắt cổng, mọi luồng tấn công trong dữ liệu phát lại đều bị bắt, nên phần bị lọt là do cổng.

Kết quả 5: cái giá của việc dùng Spark để suy luận

Đúng khu rừng đang triển khai, xuất sang các bộ máy chạy khác, cùng một bo:

Bộ máy chạy	Luồng/giây	p95 mỗi lô (ms)	mJ mỗi luồng
PySpark (đang dùng)	1,4	7515	1506
NumPy	54,8	194	11,3
ONNX Runtime	**10.092**	**1,2**	**0,087**

Lô 10 luồng, như khi triển khai, ba bộ máy cho dự đoán giống nhau 100%; mJ mỗi luồng: năng lượng tăng thêm so với lúc bo nghỉ. Mỗi lần gọi Spark tốn ≈7 giây với mọi lô tới 1.000 luồng, nên ở lô 10.000 khoảng cách chỉ còn ≈27 lần (slide dự phòng).

ONNX nhanh gấp **khoảng 7.200 lần** PySpark. Hướng tiếp theo: **huấn luyện bằng Spark, suy luận bằng ONNX/TensorRT**.



Dùng Spark để suy luận là để cùng một bộ công cụ vừa huấn luyện trên cụm vừa chạy trên thiết bị. Cái giá là JVM. Đo ở lô 10 luồng. Nếu được hỏi lô lớn hơn có khắc phục được không: mỗi lần gọi Spark tốn khoảng bảy giây dù lô lớn hay nhỏ, nên ở lô mười nghìn luồng khoảng cách còn khoảng 27 lần, nhưng ở 20 luồng mỗi giây phải chờ tám phút mới gom đủ một lô như vậy.

Kết luận

Những gì đã chỉ ra

- Tách cổng lọc và bộ phân loại ra hai bo: **2,5 → 22,4 luồng/giây**, p95 ≈7 giây.
- Năng lượng mỗi dự đoán: **3,0 J → 0,60 J**.
- Đổi lại, cổng cho lọt ≈**30%** luồng tấn công ở ngưỡng đang dùng.
- Spark là nút thắt suy luận: ONNX nhanh gấp khoảng 7.200 lần.

Hạn chế

- Hai bo, mạng Wi-Fi, dữ liệu phát lại.
- GPU chưa dùng; chưa thử tấn công né tránh.

Hướng tiếp theo

- Suy luận bằng ONNX/TensorRT.
- Lưu lượng thật; huấn luyện lại khi dữ liệu trôi.

Xin cảm ơn — kính mời câu hỏi



Nhắc lại: gấp 9 lần, năng lượng giảm 5 lần, và cái giá: khoảng 30% luồng tấn công bị cổng cho qua ở ngưỡng hiện tại.

Tài liệu tham khảo chính

[1] Sharafaldin, I., Lashkari, A. H. & Ghorbani, A. A. *Toward generating a new intrusion detection dataset and intrusion traffic characterization*. ICISSP 2018. **CICIDS2017**

[2] Shi, W. et al. *Edge computing: Vision and challenges*. IEEE Internet of Things Journal 3(5), 2016. **điện toán biên**

[3] Chandola, V., Banerjee, A. & Kumar, V. *Anomaly detection: A survey*. ACM Computing Surveys 41(3), 2009. **cổng lọc**

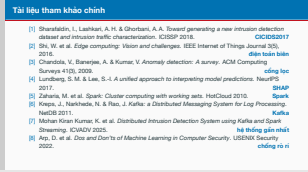
[4] Lundberg, S. M. & Lee, S.-I. *A unified approach to interpreting model predictions*. NeurIPS 2017. **SHAP**

[5] Zaharia, M. et al. *Spark: Cluster computing with working sets*. HotCloud 2010. **Spark**

[6] Kreps, J., Narkhede, N. & Rao, J. *Kafka: a Distributed Messaging System for Log Processing*. NetDB 2011. **Kafka**

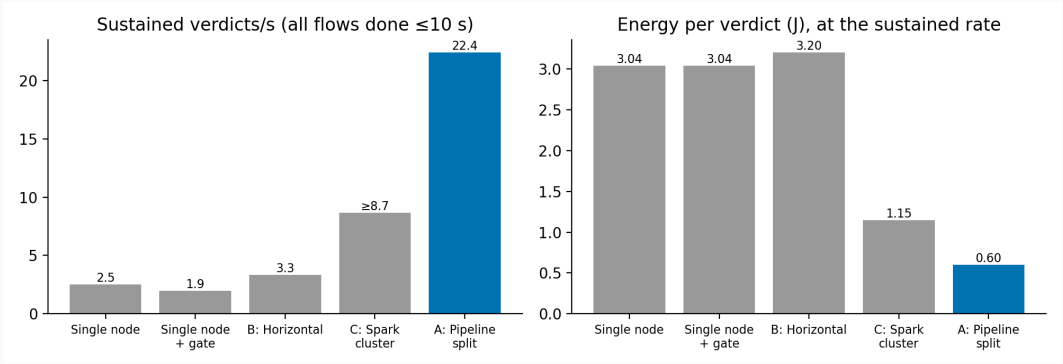
[7] Mohan Kiran Kumar, K. et al. *Distributed Intrusion Detection System using Kafka and Spark Streaming*. ICVADV 2025. **hệ thống gần nhất**

[8] Arp, D. et al. *Dos and Don'ts of Machine Learning in Computer Security*. USENIX Security 2022. **chống rò rỉ**

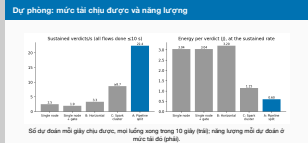


Slide dự phòng

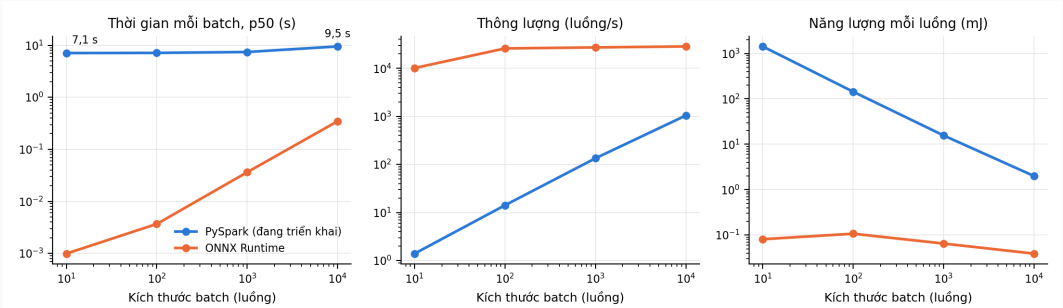
Dự phòng: mức tải chịu được và năng lượng



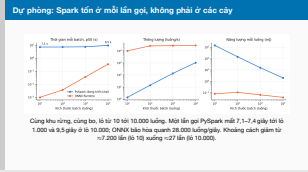
Số dự đoán mỗi giây chịu được, mọi luồng xong trong 10 giây (trái); năng lượng mỗi dự đoán ở mức tải đó (phải).



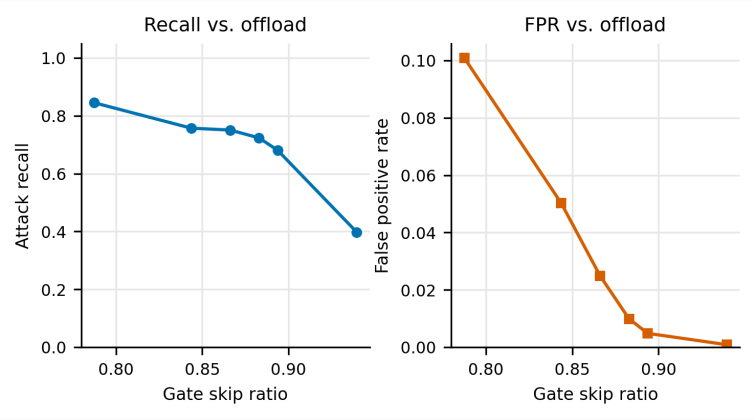
Dự phòng: Spark tốn ở mỗi lần gọi, không phải ở các cây



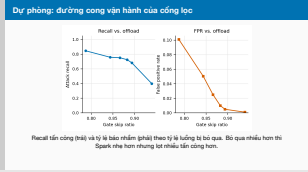
Cùng khu rừng, cùng bo, lô từ 10 tới 10.000 luồng. Một lần gọi PySpark mất 7,1–7,4 giây tới lô 1.000 và 9,5 giây ở lô 10.000; ONNX bão hòa quanh 28.000 luồng/giây. Khoảng cách giảm từ ~7.200 lần (lô 10) xuống ~27 lần (lô 10.000).



Dự phòng: đường cong vận hành của cổng lọc



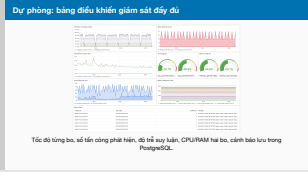
Recall tấn công (trái) và tỷ lệ báo nhầm (phải) theo tỷ lệ luồng bị bỏ qua. Bỏ qua nhiều hơn thì Spark nhẹ hơn nhưng lọt nhiều tấn công hơn.



Dự phòng: bảng điều khiển giám sát đầy đủ



Tốc độ từng bo, số tấn công phát hiện, độ trễ suy luận, CPU/RAM hai bo, cảnh báo lưu trong PostgreSQL.



Tốc độ từng bo, số tấn công phát hiện, độ trễ suy luận, CPU/RAM hai bo, cảnh báo lưu trong PostgreSQL.

Dự phòng: quy trình đo

- Máy chủ khởi động các vai trò trên Jetson qua SSH, tạo tải, thu mọi dự đoán.
- Mỗi dự đoán lưu thời điểm gửi và số dòng của luồng: độ trễ tính từ lúc gửi, *gồm cả thời gian chờ*; tỷ lệ bỏ qua và tỷ lệ bắt tấn công tính từ chính các luồng.
- Quét mức tải cho từng chế độ; một mức được coi là chịu được khi mọi luồng gửi trong 45 giây đều có kết quả trong vòng 10 giây sau đó.
- Mỗi lần chạy một consumer group Kafka riêng; mỗi tầng xử lý lô 20 luồng hoặc sau 1 giây.
- Năng lượng: `tegrastats`, 30 giây nền rồi cửa sổ tải ở mức chịu được; chế độ hai bo cộng cả hai bo.

